QU’EST-CE QUE GIT ET GITLAB ?

Git est un gestionnaire de versions. Utilisé pour créer un dépôt local et gérer les versions de fichiers.

Le gestionnaire de versions permet de garder en mémoire :

- chaque modification de chaque fichier ;
- pourquoi elle a eu lieu ;
- et par qui

GitHub est un service en ligne qui va héberger votre dépôt. Dans ce cas, on parle de **dépôt distant** puisqu’il n’est pas stocké sur une machine.

Un **dépôt local** est un entrepôt virtuel de votre projet. Il vous permet d'enregistrer les versions de votre code et d'y accéder au besoin.

Le **dépôt distant** permet de stocker les différentes versions de code afin de garder un **historique délocalisé**, c'est-à-dire un historique hébergé sur Internet ou sur un réseau.

Pourquoi créer une copie locale ?

Car un dépôt local est un clone de votre dépôt distant. C’est sur un dépôt local que toutes les modifications de code sont faîtes.

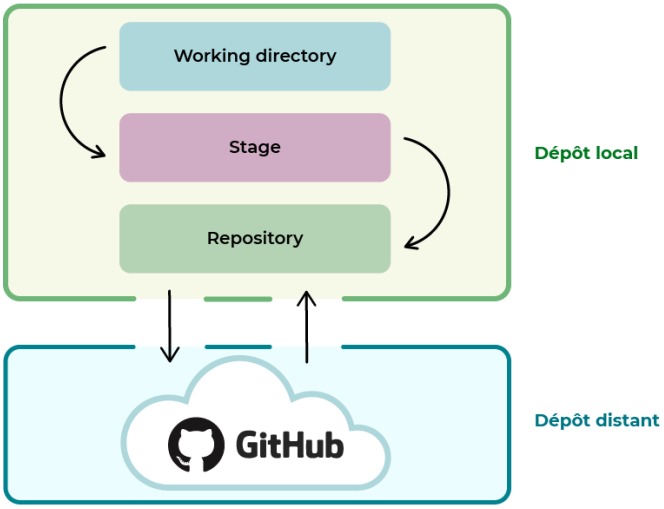
Ainsi, les dépôts sont utiles si :

- on souhaite conserver un historique de votre projet ;
- on travaille à plusieurs ;
- on souhaite collaborer à des projets open source ;
- on doit retrouver par qui a été faite chaque modification ;
- on veut savoir pourquoi chaque modification a eu lieu.

CONCLUSION :

- Un dépôt est comme un dossier qui conserve un historique des versions et des modifications d’un projet. Il est essentiel pour travailler en équipe ou collaborer à un projet open source.
- Un dépôt local est l’endroit où l’on stocke, sur sa machine, une copie d’un projet, ses différentes versions et l’historique des modifications.
- Un dépôt distant est une version dématérialisée du dépôt local, que ce soit sur Internet ou sur un réseau. Il permet de centraliser le travail des développeurs dans un projet collectif.
- Il existe plusieurs services en ligne pour héberger un dépôt distant, GitHub étant l’un des plus populaires.

1er PAS SUR GITHUB :



Le Working directory

Cette zone correspond au dossier du projet sur l'ordinateur.

Le Stage ou index

Cette zone est un intermédiaire entre le working directory et le repository. Elle représente tous les fichiers modifiés que l'on souhaite voir apparaître dans la prochaine version de code.

Le Repository

Lorsque l'on crée de nouvelles versions d'un projet, c'est dans cette zone qu'elles sont stockées.

Ces 3 zones sont donc présentes dans l'ordinateur, en local.

En-dessous, le repository GitHub, représente le dépôt distant.

Indexez les fichiers avec la commande git add :

-Une fois les fichiers créés (similaire à la création de fichier/dossier que Linux (déplacements…)), ces fichiers sont situés dans le dépôt Git. Pour créer une nouvelle version du projet, il faut maintenant indexer les fichiers.

Pour indexer ou 'stage' il faut : **$ git add index.html styles.css**

(on peut voir le status des fichiers grâce à « **git status** »)

-Une fois les fichiers indexés, il est possible de créer une version (archiver le projet en l'état) grâce à :

git commit -m "Ajout des fichiers html et css de base" (-m = description)

Envoyez le commit sur le dépôt distant avec la commande git push :

Lorsque l'on souhaite envoyer du code sur GitHub, il existe deux protocoles : SSH et HTTPS

Pour SSH il faut : générer une paire de clé SSH : **ssh-keygen -t ed25519 -C « theo.trn4@gmail.com »**

Pour récupérer la clé SSH publique, il faut : **clip < ~/.ssh/id_ed25519.pub** (copie de la clé dans le presse-papier)

Puis, ajouter la clé publique dans les paramètres de comptes GitHub

Pour relier le dépôt local au dépôt distant il faut :

-Se rendre dans le repository créer auparavant

Cliquer sur « ssh » pour ensuit copier le lien à droite

-Taper : **git remote add +le lien (ici git@github.com:theo0720/Stage01.git)**

git branch -M main

git push -u origin main

EN RESUME :

- **Le protocole SSH permet d'authentifier une personne grâce à une clé unique et personnelle.**
- **Il est possible d'ajouter une clé SSH sur un compte GitHub afin de s'authentifier plus rapidement.**
- **git add permet d'ajouter des fichiers dans l'index, qui est une zone intermédiaire dans laquelle stocker les fichiers modifiés.**
- **git commit permet de créer une nouvelle version avec les fichiers situés dans l'index.**
- **git commit -m permet de créer une nouvelle version et de préciser le message rattaché au commit.**
- **git push permet d'envoyer les modifications faites en local vers un dépôt distant.**

Git et ses systèmes de branches :

La branche principale (**main ou master**) portera **l'intégralité des modifications effectuées**. Le but n'est donc pas de réaliser les modifications directement sur cette branche, mais de **les réaliser sur d'autres branches**, et après divers tests, de les intégrer sur la branche principale.

```
theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git branch
* main

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git branch cagnotte

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git branch
  cagnotte
* main

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git checkout 'cagnotte'
Switched to branch 'cagnotte'

theot@Asus04 MINGW64 ~/Documents/PremierProjet (cagnotte)
$ git branch
* cagnotte
  main

theot@Asus04 MINGW64 ~/Documents/PremierProjet (cagnotte)
$ touch cagnotte.txt

theot@Asus04 MINGW64 ~/Documents/PremierProjet (cagnotte)
$ git add cagnotte.txt

theot@Asus04 MINGW64 ~/Documents/PremierProjet (cagnotte)
$ git commit -m "Réalisation de la partie cagnotte côté front end"
[cagnotte 6932e1b] Realisation de la partie cagnotte côté front end
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 cagnotte.txt

theot@Asus04 MINGW64 ~/Documents/PremierProjet (cagnotte)
$ git push -u origin cagnotte
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 360 bytes | 60.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'cagnotte' on GitHub by visiting:
remote:      https://github.com/theo0720/Stage01/pull/new/cagnotte
remote:
To github.com:theo0720/Stage01.git
 * [new branch]      cagnotte -> cagnotte
branch 'cagnotte' set up to track 'origin/cagnotte'.

theot@Asus04 MINGW64 ~/Documents/PremierProjet (cagnotte)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

```
theot@Asus04 MINGW64 ~/Documents/PremierProjet (cagnotte)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git branch
  cagnotte
* main

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git merge cagnotte
Updating 71427e3..6932e1b
Fast-forward
 cagnotte.txt | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 cagnotte.txt

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$
```

git branch : connaître les branches présente dans le projet

git branch nom : créer la branche « nom »

git checkout nom : basculer de branche (ici nom)

git commit -m 'Réalisation de la partie cagnotte côté front end' : pour enregistrer les évolution sur la branche « nom », il faut réaliser un commit (créer une nouvelle version de projet comprenant les évolutions réalisées sur la branche en local)

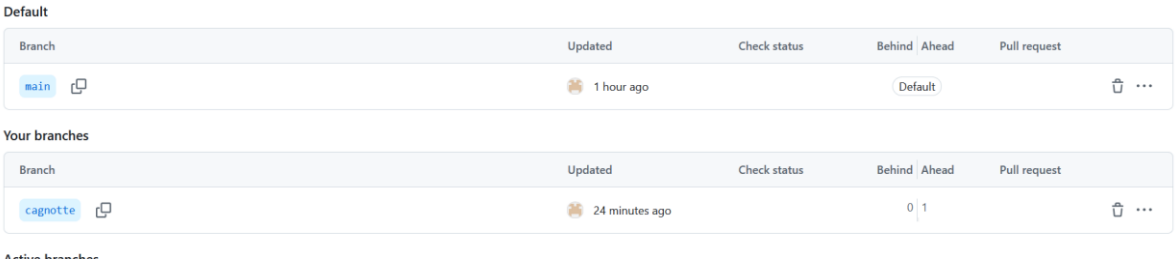
git push -u origin cagnotte : réalise le push, envoie les modifications réalisées sur le dépôt distant

Fusion du travail (branche) avec la commande « git merge »

Voilà plusieurs jours que l'on travaille sur l'évolution "cagnotte" de l'application. Après de nombreux commits et pushs, le travail semble terminé. Suite à une série de tests pour vérifier le bon fonctionnement de l'évolution "cagnotte", tout est fonctionnel.

À présent, il faut intégrer l'évolution réalisée dans la branche "cagnotte" à la branche principale "main". Pour cela : "git merge".

Cette commande doit s'utiliser à partir de la branche dans laquelle l'évolutions doit être apporté.



git checkout main: permet de se positionner dans la branche où l'on souhaite intégrer les évolutions.

git merge cagnotte : permet de fusionner la branche « cagnotte » à la branche positionné

En résumé

- Une branche est une "copie" d'un projet sur laquelle on opère des modifications de code.
- La branche main (ou anciennement master) est la branche principale d'un projet.
- git checkout permet de basculer d'une branche à une autre.
- git merge permet de fusionner deux branches.

Travaillez avec un dépôt distant

git clone +URL: pour accéder à un dépôt distant via l'adresse HTTPS ou SSH

Mettez à jour le dépôt en local :

cd nom_du_dépôt (ici 7162856-G-rez-Git-et-GitHub) : s'assurer d'être dans le bon dépôt

git pull origin main

Obtention de l'ensemble des fichiers et dossiers du repository à jour dans le répertoire courant

(test de compare and pull request) :

Retourner dans le prochain principal puis :

```
theot@Asus04 MINGW64 ~/Documents/PremierProjet/7162856-G-rez-Git-et-GitHub (main)
$ cd -
/c/Users/theot/Documents/PremierProjet

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git branch update-color

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git checkout update-color
M       cagnotte.txt
Switched to branch 'update-color'
```

```
theot@Asus04 MINGW64 ~/Documents/PremierProjet (update-color)
$ git add cagnotte.txt

theot@Asus04 MINGW64 ~/Documents/PremierProjet (update-color)
$ git status
On branch update-color
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   cagnotte.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        7162856-G-rez-Git-et-GitHub/


theot@Asus04 MINGW64 ~/Documents/PremierProjet (update-color)
$ git add 7162856-G-rez-Git-et-GitHub
warning: adding embedded git repository: 7162856-G-rez-Git-et-GitHub
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> 7162856-G-rez-Git-et-GitHub
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached 7162856-G-rez-Git-et-GitHub
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"

theot@Asus04 MINGW64 ~/Documents/PremierProjet (update-color)
$ git status
On branch update-color
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   7162856-G-rez-Git-et-GitHub
        modified:   cagnotte.txt


theot@Asus04 MINGW64 ~/Documents/PremierProjet (update-color)
$ git commit -m "Test nouvelle couleure"
[update-color 81cd07d] Test nouvelle couleure
 2 files changed, 2 insertions(+)
```

```
theot@Asus04 MINGW64 ~/Documents/PremierProjet (update-color)
$ git commit -m "Test nouvelle couleure"
[update-color 81cd07d] Test nouvelle couleure
 2 files changed, 2 insertions(+)
 create mode 160000 7162856-G-rez-Git-et-GitHub

theot@Asus04 MINGW64 ~/Documents/PremierProjet (update-color)
$ git remote add git@github.com:theo0720/Stage01.git
usage: git remote add [<options>] <name> <url>

    -f, --[no-]fetch      fetch the remote branches
    --[no-]tags           import all tags and associated objects when fetching
                          or do not fetch any tag at all (--no-tags)

    -t, --[no-]track <branch>
                          branch(es) to track

    -m, --[no-]master <branch>
                          master branch

    --[no-]mirror[=(push|fetch)]
                          set up remote as a mirror to push to or fetch from


theot@Asus04 MINGW64 ~/Documents/PremierProjet (update-color)
$ git branch -M main

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git branch -M color

theot@Asus04 MINGW64 ~/Documents/PremierProjet (color)
$ git push -u origin color
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 725 bytes | 725.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'color' on GitHub by visiting:
remote:      https://github.com/theo0720/Stage01/pull/new/color
remote:
To github.com:theo0720/Stage01.git
 * [new branch]      color -> color
branch 'color' set up to track 'origin/color'.

theot@Asus04 MINGW64 ~/Documents/PremierProjet (color)
$
```

```
theot@Asus04 MINGW64 ~/Documents/PremierProjet (color)
$ git checkout main
warning: unable to rmdir '7162856-G-rez-Git-et-GitHub': Directory not empty
branch 'main' set up to track 'origin/main'.
Switched to a new branch 'main'

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git merge color
Updating 6932e1b..81cd07d
Fast-forward
 7162856-G-rez-Git-et-GitHub | 1 +
 cagnotte.txt                | 1 +
 styles.css                  | 2 +-
 3 files changed, 3 insertions(+), 1 deletion(-)
 create mode 160000 7162856-G-rez-Git-et-GitHub

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ git push -u origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:theo0720/Stage01.git
   6932e1b..81cd07d  main -> main
branch 'main' set up to track 'origin/main'.

theot@Asus04 MINGW64 ~/Documents/PremierProjet (main)
$ |
```

En résumé

- **Sur GitHub, il est possible de récupérer l'URL d'un dépôt distant.**
- **git clone permet de copier en local un dépôt distant.**
- **git remote add permet de lier un dépôt à un "nom court", pour une plus grande facilité d'utilisation.**
- **git pull permet de dupliquer un dépôt GitHub en local.**
- **Une Pull Request permet de demander à fusionner un code sur la branche principale.**

Pratiquer et corriger des erreurs sur un dépôt local :

```
theot@Asus04 MINGW64 ~
$ mkdir Test1

theot@Asus04 MINGW64 ~
$ cd Test1

theot@Asus04 MINGW64 ~/Test1
$ git init
Initialized empty Git repository in C:/Users/theot/Test1/.git/

theot@Asus04 MINGW64 ~/Test1 (master)
$
```

```
[master (root-commit) 5237a3f] création de la branche master
 1 file changed, 1 insertion(+)
 create mode 100644 PremierFichier.txt

theot@Asus04 MINGW64 ~/Test1 (master)
$ git branch brancheTest

theot@Asus04 MINGW64 ~/Test1 (master)
$ git branch -d brancheTest
Deleted branch brancheTest (was 5237a3f).

theot@Asus04 MINGW64 ~/Test1 (master)
$ git branch -M master

theot@Asus04 MINGW64 ~/Test1 (master)
$ git branch brancheTest

theot@Asus04 MINGW64 ~/Test1 (master)
$ git branch
  brancheTest
* master

theot@Asus04 MINGW64 ~/Test1 (master)
$ git branch -d brancheTest
Deleted branch brancheTest (was 5237a3f).

theot@Asus04 MINGW64 ~/Test1 (master)
$ git branch
* master

theot@Asus04 MINGW64 ~/Test1 (master)
$
```

```
theot@Asus04 MINGW64 ~/Test1 (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   fichier.txt


theot@Asus04 MINGW64 ~/Test1 (master)
$ git stash
Saved working directory and index state WIP on master: 5237a3f création de la branch
e master

theot@Asus04 MINGW64 ~/Test1 (master)
$ git status
On branch master
nothing to commit, working tree clean

theot@Asus04 MINGW64 ~/Test1 (master)
$ git branch branche1

theot@Asus04 MINGW64 ~/Test1 (master)
$ git checkout branche1
Switched to branch 'branche1'

theot@Asus04 MINGW64 ~/Test1 (branche1)
$ git stash list
stash@{0}: WIP on master: 5237a3f création de la branche master

theot@Asus04 MINGW64 ~/Test1 (branche1)
$   git stash apply
On branch branche1
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   fichier.txt


theot@Asus04 MINGW64 ~/Test1 (branche1)
$   git stash apply stash@{0}
On branch branche1
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   fichier.txt


theot@Asus04 MINGW64 ~/Test1 (branche1)
$ |
```

$ git log

git reset --hard HEAD^ : supprime le dernier commit de la branche principale

En résumé

- git branch -d permet de supprimer une branche.
- git status permet de voir l’état des fichiers.
- git stash enregistre les modifications non indexées pour une utilisation ultérieure.
- git log affiche l'historique des commits réalisés sur la branche courante.
- git reset --hard HEAD^ permet de réinitialiser l'index et le répertoire de travail à l'état du dernier commit.
- git commit --amend permet de sélectionner le dernier commit pour y effectuer des modifications.

En résumé

- git revert HEAD^ permet d'annuler un commit en créant un nouveau commit.
- git reset est une commande puissante. Elle peut être appliquée de 3 façons différentes (--soft; --mixed; --hard).
- La commande git merge produit un conflit si une même ligne a été modifiée plusieurs fois. Dans ce cas, il faut indiquer à Git quelle ligne conserver.
- git reset permet de revenir à l'état précédent sans créer un nouveau commit.
- git revert permet de revenir à l'état précédent en créant un nouveau commit.

En résumé

- git log affiche l'historique des commits réalisés sur la branche courante.
- git reflog est identique à git log. Cette commande affiche également toutes les actions réalisées en local.
- git checkout un_identifiant_SHA-1 permet de revenir à une action donnée.
- git blame permet de savoir qui a réalisé telle modification dans un fichier, à quelle date, ligne par ligne.
- git cherry-pick un_identifiant_SHA-1 un_autre_identifiant_SHA-1 permet de sélectionner un commit et de l'appliquer sur la branche actuelle.
- SHA est un identifiant pour les commits et autres actions gardés en mémoire par Git. Il représente un grand code qui permet de revenir à un commit exact. C’est l’identification de l’action